

# **ЛАБОРАТОРНАЯ РАБОТА №7**

**Дискретное логарифмирование в конечном поле**

Хамза хуссен

# Содержание

|          |                                                                                              |           |
|----------|----------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Цель работы</b>                                                                           | <b>3</b>  |
| <b>2</b> | <b>Задание</b>                                                                               | <b>4</b>  |
| <b>3</b> | <b>Теоретическое введение</b>                                                                | <b>5</b>  |
| 3.1      | Алгоритм, реализующий $\rho$ -метод Полларда для задач дискретного логарифмирования. . . . . | 5         |
| 3.2      | Пример . . . . .                                                                             | 6         |
| <b>4</b> | <b>Выполнение лабораторной работы</b>                                                        | <b>7</b>  |
| <b>5</b> | <b>Выводы</b>                                                                                | <b>10</b> |
|          | <b>Список литературы</b>                                                                     | <b>11</b> |

# 1 Цель работы

Реализация алгоритм,  $\rho$ -метод Полларда для вычисления дискретного логарифма.

## 2 Задание

1. Реализовать алгоритм,  $\rho$ -метод Полларда для вычисления дискретного логарифма.

## 3 Теоретическое введение

Обозначим  $F_p = \mathbb{Z}/p\mathbb{Z}$ ,  $p$  - простое целое число и назовем конечным полем из  $p$  элементов. Задача дискретного логарифмирования в конечном поле  $F_p$  формулируется так: для данных целых чисел  $a$  и  $b$ ,  $a > 1, b > 0$ , найти логарифм - такое целое число  $x$ , что  $a^x = b \pmod{p}$  (если такое число существует). По аналогии с вещественными числами используется обозначение  $x = \log_a b$

### 3.1 Алгоритм, реализующий $\rho$ -метод Полларда для задач дискретного логарифмирования.

Вход. Простое число  $p$ , число  $a$  порядка  $g$  по модулю  $p$ , целое число  $b$ ,  $1 < b < p$ ; отображение  $f$ , обладающее сжимающими свойствами и сохраняющее вычислимость логарифма.

Выход. Показатель  $x$ , для которого  $a^x = b \pmod{p}$ , если такой показатель существует.

1. Выбрать произвольные целые числа  $u, v$  и положить  $s \leftarrow a^u b^v \pmod{p}$ ,  $d \leftarrow s$ .
2. Выполнять  $s \leftarrow f(s) \pmod{p}$ ,  $d \leftarrow f(f(d)) \pmod{p}$ , вычисляя при этом логарифмы для  $s$  и  $d$  как линейные функции от  $x$  по модулю  $g$ , до получения равенства  $s = d \pmod{p}$ .
3. Приравняв логарифмы для  $s$  и  $d$ , вычислить логарифм  $x$  решением сравнения по модулю  $g$ . Результат:  $x$  или «Решений нет».

## 3.2 Пример

Пример. Решим задачу дискретного логарифмирования  $10^* = 64 \pmod{107}$ , используя р-Метод Полларда. Порядок числа 10 по модулю 107 равен 53. Выберем  $64c \pmod{107}$  при  $c \geq 53$ . Пусть  $u = 2$ ,  $v = 2$ . Результаты вычислений запишем в таблицу:

| Номер шага | c  | $\log_a c$ | d  | $\log_a d$ |
|------------|----|------------|----|------------|
| 0          | 4  | $2+2x$     | 4  | $2+2x$     |
| 1          | 40 | $3+2x$     | 76 | $4+2x$     |
| 2          | 79 | $4+2x$     | 56 | $5+3x$     |
| 3          | 27 | $4+3x$     | 75 | $5+5x$     |
| 4          | 56 | $5+3x$     | 3  | $5+7x$     |
| 5          | 53 | $5+4x$     | 86 | $7+7x$     |
| 6          | 75 | $5+5x$     | 42 | $8+8x$     |
| 7          | 92 | $5+6x$     | 23 | $9+9x$     |
| 8          | 3  | $5+7x$     | 53 | $11+9x$    |
| 9          | 30 | $6+7x$     | 92 | $11+11x$   |
| 10         | 86 | $7+7x$     | 30 | $12+12x$   |
| 11         | 47 | $7+8x$     | 47 | $13+13x$   |

Приравниваем логарифмы, полученные на 11-м шаге:  $7+8x = 13+13x \pmod{53}$ . Решая сравнение первой степени, получаем:  $x = 20 \pmod{53}$ .

Проверка:  $10^{20} = 64 \pmod{107}$ .

## 4 Выполнение лабораторной работы

```
[38]: from math import gcd

def extended_gcd(a, b):
    """
    Расширенный алгоритм Евклида
    возвращает (g, x, y) такое что

    ax + by = g = gcd(a,b)
    """
    if b == 0:
        return (a, 1, 0)

    g, x1, y1 = extended_gcd(b, a % b)

    x = y1
    y = x1 - (a // b) * y1

    return (g, x, y)
```

Рисунок 4.1: extended\_gcd

```
[39]: def mod_inverse(a, m):
    """
    Обратный элемент a^{-1} mod m
    """
    g, x, _ = extended_gcd(a, m)

    if g != 1:
        return None

    return x % m
```

Рисунок 4.2: mod\_inverse

```
[40]: def f(value, u, v, p, a, b, r):
      """
      Отображение f из алгоритма.

      Поддерживается инвариант:
      value = a^u * b^v (mod p)
      """

      subset = value % 3

      if subset == 0:

          # value ← value * b
          value = (value * b) % p
          v = (v + 1) % r

      elif subset == 1:

          # value ← value^2
          value = (value * value) % p
          u = (2 * u) % r
          v = (2 * v) % r

      else:

          # value ← value * a
          value = (value * a) % p
          u = (u + 1) % r

      return value, u, v
```

Рисунок 4.3: отображение f

```
[41]: def pollard_rho_discrete_log(p, a, b, r):
      """
      Реализация алгоритма из конспекта.

      Вход:
      p — простое число
      a — элемент порядка r по mod p
      b — число
      r — порядок a

      Ищем x:
      a^x ≡ b (mod p)
      """

      # -----
      # ШАГ 1
      # выбрать u, v и вычислить
      # c = a^u * b^v (mod p)
      # d = c
      # -----

      u_c = 2
      v_c = 2

      c = (pow(a, u_c, p) * pow(b, v_c, p)) % p

      d = c
      u_d = u_c
      v_d = v_c

      step = 0

      print("step | c | log_a(c) | d | log_a(d)")
```

Рисунок 4.4: pollard\_rho\_discrete\_log



```

print("step | c | log_a(c) | d | log_a(d)")
print("-----")

while True:

    print(
        step,
        "|",
        c,
        "|",
        f"{u_c}+{v_c}x",
        "|",
        d,
        "|",
        f"{u_d}+{v_d}x",
    )

    # -----
    # ШАГ 2
    # c ← f(c)
    # d ← f(f(d))
    # -----

    # --- обновление c ---
    c, u_c, v_c = f(c, u_c, v_c, p, a, b, r)

    # --- обновление d два раза ---
    d, u_d, v_d = f(d, u_d, v_d, p, a, b, r)
    d, u_d, v_d = f(d, u_d, v_d, p, a, b, r)

    step += 1

    # -----
    # проверяем столкновение
    # -----

    if c == d:
        print("\nCollision found")
        break

```

Рисунок 4.5: pollard\_rho\_discrete\_log

```

[42]: p = 107
      a = 10
      b = 64
      r = 53

      x = pollard_rho_discrete_log(p, a, b, r)

      print("\nResult x =", x)

      print("Check:")
      print(pow(a, x, p))

```

Рисунок 4.6: test

```

Result x = 20
Check:
64

```

Рисунок 4.7: result

## 5 Выводы

Реализовать алгоритм,  $\rho$ -метод Полларда для вычисления дискретного логарифма.

## Список литературы

[https://en.wikipedia.org/wiki/Pollard%27s\\_rho\\_algorithm](https://en.wikipedia.org/wiki/Pollard%27s_rho_algorithm)